Test Bench

1 Introduction

A Verilog testbench is a simulation-only procedural module used to exercise a design under test (DUT). It supplies input patterns (test vectors), may generate a clock and reset, and observes the DUT's outputs. Testbenches help check behavior before synthesis or hardware implementation; they are not part of the synthesized circuit.

The testbench connects to the DUT's inputs and outputs. In traditional Verilog, testbench signals that are assigned inside procedural blocks such as `initial` or `always` are declared as `reg`; signals driven by the DUT are usually declared as `wire`. An `initial` block runs once, while an `always` block repeats, making it useful for clock generation.

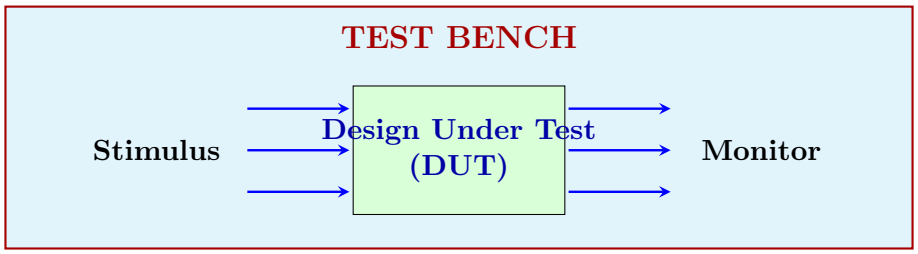


Figure 1: A testbench applies stimulus to the DUT and observes its outputs.

2 Writing a Testbench

A useful starting template has three parts: declare the DUT interface, instantiate the DUT, then initialize inputs and observe outputs. The testbench is a separate simulation module and does not need to be synthesizable.

2.1 A Simple Gate-Level Example

The following example describes a small gate network and a testbench that applies input combinations at five-time-unit intervals. The testbench uses `$monitor` to report signal values whenever one of its listed values changes.

```
module example (A, B, C, D, E, F, Y);
  input A, B, C, D, E, F;
  output Y;
  wire t1, t2, t3;

  nand #1 G1 (t1, A, B);
  and  #2 G2 (t2, C, ~D);
  nor  #1 G3 (t3, E, F);
  nand #1 G4 (Y, t1, t2, t3);
endmodule

module testbench;
  reg A, B, C, D, E, F;
  wire Y;
  example DUT (A, B, C, D, E, F, Y);

  initial begin
    $monitor ($time, " A=%b, B=%b, C=%b, D=%b, E=%b, F=%b, Y=%b",
              A, B, C, D, E, F, Y);
    #5 A=1; B=1; C=0; D=1; E=0; F=0;
    #5 A=0; B=0; C=1; D=1; E=0; F=0;
    #5 A=1; C=0;
    #5 F=1;
    #5 $finish;
  end
endmodule
```

Listing 1: Gate-level design and testbench

2.2 Simulator Directives

The system tasks below help display values, end a run, and save waveform data in a Value Change Dump (VCD) file. The waveform can be opened in a viewer after simulation.

- `$display(format, ...)` prints a line immediately. Its format string supports conversions such as `%b` for binary and `%h` for hexadecimal.
- `$monitor(format, ...)` prints when any listed value changes; it is useful for event-driven observation.
- `$finish` terminates the simulation.
- `$dumpfile("name.vcd")` selects the waveform output file.
- `$dumpvars(level, scope)` selects signals to record. Level `0` includes the selected scope and its hierarchy; for example, `$dumpvars(0, testbench)` records the testbench hierarchy.
- `$dumpall` writes the current values of the selected signals.
- `$dumpoff` pauses waveform recording, and `$dumpon` resumes it.
- `$dumplimit(size)` sets a maximum VCD file size.

2.3 Synchronous Sequential Circuits

Sequential designs need a clock stimulus. A simple clock toggles every five time units, producing a ten-time-unit period:

```
initial clk = 1'b0;
always #5 clk = ~clk;
```

The timescale directive defines the units used by delays such as `#5`. The following 2-bit equality checker and testbench illustrate stimulus, waveform dumping, and output monitoring. The checker asserts `z` when both input vectors match.

```
`timescale 1ns / 100ps
module comparator (x, y, z);
  input [1:0] x, y;
  output z;

  assign z = ( x[0] &  y[0] &  x[1] &  y[1]) |
             (~x[0] & ~y[0] &  x[1] &  y[1]) |
             (~x[0] & ~y[0] & ~x[1] & ~y[1]) |
             ( x[0] &  y[0] & ~x[1] & ~y[1]);
endmodule

module testbench;
  reg [1:0] x, y;
  wire z;
  comparator C2 (.x(x), .y(y), .z(z));

  initial begin
    $dumpfile("comp.vcd");
    $dumpvars(0, testbench);
    x = 2'b01; y = 2'b00;
    #10 x = 2'b10; y = 2'b10;
    #10 x = 2'b01; y = 2'b11;
  end

  initial
    $monitor("t=%0d x=%2b y=%2b z=%b", $time, x, y, z);
endmodule
```

Listing 2: 2-bit equality checker and testbench

3 Full-Adder Testbenches

A full adder produces a sum bit and a carry-out bit from inputs `a`, `b`, and carry-in `c`. Its Boolean equations are shown here; the following testbenches demonstrate monitoring, display-on-demand, looping over input vectors, and checking expected results.

```
module full_adder (s, co, a, b, c);
  input a, b, c;
  output s, co;

  assign s  = a ^ b ^ c;
  assign co = (a & b) | (b & c) | (c & a);
endmodule
```

Listing 3: Full-adder logic

3.1 Monitoring Outputs

Unlike `$monitor`, which reports changes automatically, `$display` prints only when its statement executes. This example prints the result after each input update.

```
module testbench;
  reg a, b, c;
  wire sum, cout;
  full_adder FA (sum, cout, a, b, c);

  initial begin
    $monitor ($time, " a=%b, b=%b, c=%b, sum=%b, cout=%b",
              a, b, c, sum, cout);
    #5 a=0; b=0; c=1;
    #5 b=1;
    #5 a=1;
    #5 a=0; b=0; c=0;
    #5 $finish;
  end
endmodule
```

Listing 4: Stimulus with automatic monitoring

```
module testbench;
  reg a, b, c;
  wire sum, cout;
  full_adder FA (sum, cout, a, b, c);

  initial begin
    a=0; b=0; c=1; #5;
    $display("T=%2d, a=%b, b=%b, c=%b, sum=%b, cout=%b",
             $time, a, b, c, sum, cout);
    b=1; #5;
    $display("T=%2d, a=%b, b=%b, c=%b, sum=%b, cout=%b",
             $time, a, b, c, sum, cout);
    a=1; #5;
    $display("T=%2d, a=%b, b=%b, c=%b, sum=%b, cout=%b",
             $time, a, b, c, sum, cout);
    a=0; b=0; c=0; #5;
    $display("T=%2d, a=%b, b=%b, c=%b, sum=%b, cout=%b",
             $time, a, b, c, sum, cout);
    #5 $finish;
  end
endmodule
```

Listing 5: Explicit display after each input update

3.2 Iterating Through Input Vectors

Each three-bit value of `i` is assigned to inputs `a`, `b`, and `c`, so the loop checks all eight possible vectors.

```
module testbench;
  reg a, b, c;
  integer i;
  wire sum, cout;
  full_adder FA (sum, cout, a, b, c);

  initial begin
    for (i=0; i<8; i=i+1) begin
      {a,b,c} = i;
      #5;
      $display("T=%2d, a=%b, b=%b, c=%b, sum=%b, cout=%b",
               $time, a, b, c, sum, cout);
    end
    #5 $finish;
  end
endmodule
```

Listing 6: Full-adder test using a for loop

3.3 Automatic Verification

This example compares the DUT outputs against the expected values for three selected input vectors. The flag remains 1 if all three checks pass and is cleared to 0 if any check fails.

```
module fulladder (a, b, c, s, cout);
  input a, b, c;
  output s, cout;
  assign s = a ^ b ^ c;
  assign cout = (a & b) | (b & c) | (c & a);
endmodule

module fulladder_test;
  reg a, b, c;
  wire s, cout;
  integer correct;
  fulladder FA (a, b, c, s, cout);

  initial begin
    correct = 1;
    #5 a=1; b=1; c=0; #5;
    if ((s != 0) || (cout != 1)) correct = 0;
    #5 a=1; b=1; c=1; #5;
    if ((s != 1) || (cout != 1)) correct = 0;
    #5 a=0; b=1; c=0; #5;
    if ((s != 1) || (cout != 0)) correct = 0;
    #5 $display("%d", correct);
  end
endmodule
```

Listing 7: Checking selected full-adder results

4 Sequential-Circuit Examples

4.1 4-Bit Shift Register

This shift register has an active-low asynchronous clear. On each rising clock edge, the serial input advances through the four stages; the final stage is the output.

```
module shiftreg_4bit (clock, clear, A, E);
  input clock, clear, A;
  output reg E;
  reg B, C, D;

  always @(posedge clock or negedge clear) begin
    if (!clear) begin
      B <= 0; C <= 0; D <= 0; E <= 0;
    end else begin
      E <= D;
      D <= C;
      C <= B;
      B <= A;
    end
  end
endmodule
```

Listing 8: 4-bit shift-register design

```
module shift_test;
  reg clk, clr, in;
  wire out;
  shiftreg_4bit SR (clk, clr, in, out);

  initial begin
    clk = 1'b0;
    clr = 1'b1;
    #2 clr = 0;
    #5 clr = 1;
  end

  always #5 clk = ~clk;

  initial begin
    #2;
    repeat (2) begin
      #10 in=0; #10 in=0; #10 in=1; #10 in=1;
    end
  end

  initial begin
    $dumpfile("shifter.vcd");
    $dumpvars(0, shift_test);
    #200 $finish;
  end
endmodule
```

Listing 9: Shift-register testbench

4.2 7-Bit Binary Counter

The counter below increments on each falling clock edge and clears when `clear` is high. The width is expressed as `N` bits, so the seven-bit instance uses `N=7` and a range of `[N-1:0]`.

```
module counter (clear, clock, count);
  parameter N = 7;
  input clear, clock;
  output reg [N-1:0] count;

  always @(negedge clock)
    if (clear)
      count <= 0;
    else
      count <= count + 1;
endmodule

module test_counter;
  reg clk, clr;
  wire [6:0] out;
  counter CNT (clr, clk, out);

  initial clk = 1'b0;
  always #5 clk = ~clk;

  initial begin
    clr = 1'b1;
    #15 clr = 1'b0;
    #200 clr = 1'b1;
    #10 $finish;
  end

  initial begin
    $dumpfile("counter.vcd");
    $dumpvars(0, test_counter);
    $monitor($time, " Count: %d", out);
  end
endmodule
```

Listing 10: Parameterized counter and testbench

5 Random Test Vectors

The `$random` system function generates pseudo-random values. An optional seed can make a run reproducible; here, the integer seed starts at 15. The adder and testbench below generate five pairs of eight-bit inputs.

```
module adder (out, cout, a, b);
  input [7:0] a, b;
  output [7:0] out;
  output cout;

  assign #5 {cout, out} = a + b;
endmodule
```

Listing 11: 8-bit adder with delayed outputs

```
module test_adder;
  reg [7:0] a, b;
  wire [7:0] sum;
  wire cout;
  integer myseed;
  adder ADD (sum, cout, a, b);

  initial myseed = 15;

  initial begin
    repeat (5) begin
      a = $random(myseed);
      b = $random(myseed);
      #10;
      $display("T: %3d, a: %h, b: %h, sum: %h",
               $time, a, b, sum);
    end
  end
endmodule
```

Listing 12: Randomized adder testbench